

Testplan: Traivo Databasstruktur

Datum: 2026-06-11
Version: 1.0
Baserat på: Mats beskrivning av databasstruktur (session 2026-06-11)
Syfte: Verifiera att databasstrukturen matchar Mats logiska modell och fungerar korrekt under alla förhållanden

1. Förberedelser

1.1 Testmiljö

- Dedikerad testdatabas (INTE produktion)
- Samma Prisma-schema som produktionsmiljön
- Seedade testdata (se 1.2)
- Möjlighet att nollställa mellan testkörningar

1.2 Testdata - Realistiska scenarier

Tenant-uppsättning

Tenant	Beskrivning
Företag A	Städföretag – 3 kunder, 50 objekt, 200 metadatafält, 10 orderkoncept
Företag B	Avfallshantering – 5 kunder, 300 objekt (hierarkiska kluster), 1000 metadatafält, 30 orderkoncept
Företag C	Tomt företag – inga data (edge case)

Kunder (Företag B - huvudscenario)

Kund	Kundnummer	Beskrivning
Hemköp Södertälje	K001	3 kluster, 20 objekt
BRF Storgatan	K002	1 kluster, 5 objekt
ICA Maxi	K003	2 kluster, 100 objekt (stresstest)
Kommun Södertälje	K004	Delar kluster med K001 (N:M-test)
Test AB	K005	Ingen koppling till objekt (edge case)

Objekt-hierarki (Hemköp Södertälje)

```

Hemköp Huvudkontor (010)
├─ Avfallsrum A (001) – lon/lat: 59.1955, 17.6253
│   Metadata: gatuadress="Storgatan 22", ort="Södertälje",
│             objekttyp="avfallsrum", antal_matavfallskärl=10,
│             önskad_leveransdag="torsdag", foto=bild.jpg
├─ Avfallsrum B (002) – lon/lat: 59.1955, 17.6254 (inom 30m!)
│   Metadata: antal_matavfallskärl=5
├─ Lastbrygga (003) – lon/lat: 59.1960, 17.6260 (utanför 30m)
│   Metadata: objekttyp="lastbrygga"
└─ Avfallsrum C (004) – INGEN geo-position (edge case)
  
```

Artiklar

Artikel	Nr	Typ	Haka-på villkor	Antal-logik
Tvätta rum	A100	Struktur	objekttyp="avfallsrum"	Från metadata
Tvätta kärl	A101	Under-artikel (A100)	-	antal_matavfallskärl
Tvätta golv	A102	Under-artikel (A100)	-	Fast: 1
Dörrkontroll	A103	Enkel	objekttyp="avfallsrum"	Fast: 1, kräver foto
Kranbilslyft	A104	Enkel	objekttyp="lastbrygga"	Fast: 1, kräver kranbil
Leveransartikel	A105	Fysisk	-	Fast: 1, lagerplats geo

Orderkoncept

Koncept	Kund	Artiklar	Period	Repetition
OC-001	K001	A100 (tvätta rum)	Jan-Feb, Maj-Jun, Aug-Sep	Årlig, slut 2028
OC-002	K002	A103 (dörrkontroll)	Varannan månad	Löpande
OC-003	K003	A100, A104	Hela året, torsdagar 09-11:30	Veckovis
OC-004	K001	A105 (leverans)	Engångs	-

Utförare

Utförare	Typ	Utförandety	Team
Erik	Person	Städ	Team Alpha
Anna	Person	Städ	Team Alpha
Kranbil-01	Utrustning	Kranbil	Team Beta
Johan	Person	Kranbil	Team Beta

2. Strukturella tester

A. Schema-validering

Test A1: Alla tabeller existerar

FÖR VARJE tabell i [Kund, Artikel, Objekt, MetadataDefinition, MetadataVärde, Orderkoncept, Uppgift, Utförandety, Utförare, Team, Artikelty, Metadataty, Ikon]:
 VERIFIERA: Tabellen existerar i databasen
 VERIFIERA: Tabellen har korrekta fält (se checklista i verifieringsdokumentet)

Acceptanskriterie: Alla 13+ tabeller existerar med korrekta fält.

Test A2: Alla relationer (foreign keys) existerar

VERIFIERA relationer:

- Kund ↔ Objekt (N:M via kopplingsstabelle)
- Objekt → Objekt (self-ref, parent_id)
- MetadataVärde → MetadataDefinition (FK)
- MetadataVärde → Objekt (FK)
- Orderkoncept → Kund (FK)
- Uppgift → Orderkoncept (FK)
- Uppgift → Artikel (FK)
- Uppgift → Objekt (FK)
- Artikel → Artikel (self-ref, strukturartikel)
- Artikel → Utförandety (FK)
- MetadataDefinition → MetadataDefinition (self-ref, hierarki)
- Utförare ↔ Team (N:M)

Acceptanskriterie: Alla FK-constraints existerar och är korrekt konfigurerade.

Test A3: Constraints verifieras

FÖR VARJE tabell:

- VERIFIERA: NOT NULL constraints på obligatoriska fält
- VERIFIERA: UNIQUE constraints (kundnummer, artikelnummer, metadata-referensnummer)
- VERIFIERA: CHECK constraints (om applicerbart)
- VERIFIERA: DEFAULT values (t.ex. status, timestamps)

Test A4: Index verifieras

VERIFIERA att följande index existerar:

- tenant_id på ALLA tabeller
- Geospatial index på objektens lon/lat
- Kompositindex: (tenant_id, kund_id) på orderkoncept
- Kompositindex: (tenant_id, status, leveranstid) på uppgifter
- Index på metadata_definition_id + objekt_id på metadatavärden
- Index på artikel_parent_id (strukturartiklar)

B. Multi-tenant isolering

Test B1: Data-isolering mellan företag

SCENARIO: Företag A och B har varsitt kundregister

STEG:

1. Skapa kund "TestKund" i Företag A
2. Skapa kund "TestKund" i Företag B (samma namn!)
3. VERIFIERA: Query med tenant_id=A returnerar BARA A:s kund
4. VERIFIERA: Query med tenant_id=B returnerar BARA B:s kund
5. VERIFIERA: Utan tenant_id-filter ☐ FEL eller alla (beroende på design)

Upprepas för: Artiklar, Objekt, Metadatadefinitioner, Metadatavärden, Orderkoncept, Uppgifter, Utförare

Test B2: Tenant-filter på ALLA queries

FÖR VARJE API-endpoint / databasfunktion:

VERIFIERA: Query innehåller WHERE tenant_id = ?

VERIFIERA: Det är OMÖJLIGT att hämta data utan tenant_id

VERIFIERA: Ingen endpoint returnerar data från annan tenant

Test B3: Cross-tenant referensintegritet

SCENARIO: Försök skapa orderkoncept i Tenant A som refererar kund i Tenant B

FÖRVÄNTAT: Felmeddelande / blockerat

SCENARIO: Försök tilldela uppgift i Tenant A till utförare i Tenant B

FÖRVÄNTAT: Felmeddelande / blockerat

Test B4: Cascade inom tenant-gränser

SCENARIO: Ta bort ett objekt i Tenant A

VERIFIERA: Metadatavärden för objektet tas bort/markeras

VERIFIERA: INGA objekt/metadata i Tenant B påverkas

C. Relationsintegritet

Test C1: CRUD per entitet

FÖR VARJE entitet [Kund, Artikel, Objekt, ...]:
 CREATE: Skapa med alla obligatoriska fält → LYCKAS
 CREATE: Skapa utan obligatoriskt fält → MISSLYCKAS med tydligt fel
 READ: Läs tillbaka → data matchar
 UPDATE: Ändra fält → sparas korrekt
 DELETE: Ta bort → korrekt cascade-beteende

Test C2: Kund ↔ Objekt (N:M)

STEG:

1. Koppla Kund K001 till Kluster 010
2. Koppla Kund K004 till Kluster 010 (samma kluster!)
3. VERIFIERA: K001 ser Kluster 010 i sin vy
4. VERIFIERA: K004 ser Kluster 010 i sin vy
5. VERIFIERA: Kluster 010 listar båda kunderna
6. Ta bort kopplingen K001 ↔ 010
7. VERIFIERA: K004 fortfarande kopplad till 010
8. VERIFIERA: Kluster 010 fortfarande existerar

Test C3: Objekt-hierarki (förälder/barn)

STEG:

1. Skapa rot-objekt (utan förälder)
2. Skapa barn-objekt med förälder
3. Skapa barnbarn-objekt
4. VERIFIERA: Trädstrukturen kan traverseras
5. Försök ta bort förälder med barn → korrekt beteende (blockera ELLER cascade)
6. Försök skapa cirkulär referens → FEL

Test C4: Metadata-hierarki (strukturerade fält)

STEG:

1. Skapa metadatadefinition "Kontakt" (huvud)
2. Skapa underfält: Namn, Titel, Telefon, E-post
3. Koppla "Kontakt" till objekt
4. VERIFIERA: Alla underfält följer med
5. Uppdatera "Namn" → logg skapas (vem + när)
6. VERIFIERA: Arv neråt i kluster fungerar

Test C5: Strukturartiklar

STEG:

1. Skapa huvudartikel A100 "Tvätta rum"
2. Skapa underartiklar A101, A102
3. Koppla underartiklar till A100
4. VERIFIERA: A100 returnerar alla underartiklar
5. Skapa orderkoncept med A100
6. VERIFIERA: Uppgifter skapas för ALLA underartiklar

Test C6: Orphaned records prevention

SCENARIO: Ta bort artikel som refereras av aktivt orderkoncept
FÖRVÄNTAT: Blockerat (referential integrity)

SCENARIO: Ta bort metadatadefinition som har värden
FÖRVÄNTAT: Blockerat ELLER cascade med varning

SCENARIO: Ta bort kund som har aktiva orderkoncept
FÖRVÄNTAT: Blockerat

D. Prestanda

Test D1: Stora datamängder

FÖRBERED:

- 100 000 objekt i ett kluster
- 50 metadatafält per objekt (= 5 000 000 metadatavärden)
- 1 000 orderkoncept
- 500 000 uppgifter

TESTA:

- Hämta alla objekt i ett kluster: < 2 sekunder
- Söka objekt via metadatavillkor: < 1 sekund
- Generera uppgifter från orderkoncept (100 objekt): < 5 sekunder
- Hämta grovplaneringslistan (1 månad framåt): < 3 sekunder
- Geospatial sökning (30m radie): < 1 sekund

Test D2: Index-verifiering

FÖR VARJE kritisk query:

KÖR EXPLAIN ANALYZE

VERIFIERA: Index används (inte seq scan)

VERIFIERA: Query plan är optimal

KRITISKA QUERIES:

- Hämta objekt med specifikt metadatavillkor
- Hämta uppgifter per kund, sorterade på leveranstid
- Geospatial sökning inom 30m
- Hämta alla orderkoncept per kund med status
- Hämta metadatavärden per objekt

Test D3: N+1 Query Problem

SCENARIO: Hämta orderkoncept med alla relaterade data

VERIFIERA: Ingen N+1 – alla relationer hämtas med JOIN/include

SCENARIO: Hämta objekt-träd med metadata

VERIFIERA: Inte en query per barn-objekt

SCENARIO: Hämta uppgiftslista med artiklar och metadata

VERIFIERA: Effektiv batch-hämtning

Test D4: Samtidiga operationer (concurrency)

SCENARIO: Två utförare uppdaterar samma metadatafält samtidigt

VERIFIERA: Race condition hanteras (optimistic locking?)

SCENARIO: Orderkoncept genererar uppgifter medan utförare uppdaterar metadata

VERIFIERA: Konsekvent resultat

E. Dataflöden

Test E1: Komplett flöde - Engångsorder

STEG:

1. Skapa kund K001 (synk med Fortnox)
2. Skapa objekt med metadata (gatuadress, antal_kärl=10)
3. Skapa artikel A100 med haka-på: objekttyp="avfallsrum", antal=metadata
4. Skapa orderkoncept OC-001: kund=K001, leverans=jan-feb
5. Bekräfta orderkoncept på kluster
6. VERIFIERA: Uppgifter skapas för alla matchande objekt
7. VERIFIERA: Antal hämtas från metadata (10 kärl)
8. VERIFIERA: Leveranstid sätts till jan-feb
9. VERIFIERA: Uppgifter syns i grovplaneringslistan
10. Tilldela utförare
11. Utförare slutför uppgift + uppdaterar antal (10→12)
12. VERIFIERA: Metadatavärdet uppdateras till 12
13. VERIFIERA: Framtida uppgifter uppdateras till 12
14. VERIFIERA: Logg: vem uppdaterade + tidsstämpel

Test E2: Komplett flöde - Repeterande orderkoncept

STEG:

1. Skapa orderkoncept med 3 leveransperioder per år, slutdatum 2028
2. VERIFIERA: Uppgifter genereras för rullande 18 månader
3. VERIFIERA: Inte uppgifter för hela perioden till 2028
4. Vänta tills ny period → VERIFIERA: nya uppgifter genereras
5. Ändra slutdatum till 2027 → VERIFIERA: framtida uppgifter efter 2027 tas bort

Test E3: Haka-på logik

SCENARIO: Artikel med villkor objekttyp="avfallsrum"

- Objekt MED objekttyp="avfallsrum" → artikel hakar fast ✓
- Objekt UTAN objekttyp → artikel hakar INTE fast ✓
- Objekt med objekttyp="lastbrygga" → artikel hakar INTE fast ✓

SCENARIO: Artikel med FLERA villkor (objekttyp="avfallsrum" OCH fraktion="matavfall")

- Objekt med båda → hakar fast ✓
- Objekt med bara ett → hakar INTE fast ✓

Test E4: Kvantitetsresolution (3-steps)

STEG:

1. Orderkoncept anger antal=5
2. Objekt har metadatafält antal_kärl=10
3. VERIFIERA: Uppgift skapas med antal=10 (metadata övertar orderkoncept)
4. Utförare ändrar till 12 (tillåtet)
5. VERIFIERA: Uppgift uppdateras till 12

EDGE CASES:

- Metadatafält saknas → antal=5 (från orderkoncept)
- Metadatafält finns men tomt → antal=5 (fallback)
- Utförare försöker ändra men ej tillåtet → blockeras

Test E5: 30-meters jobbgruppering

STEG:

1. Objekt A: lon=59.1955, lat=17.6253 (Kund K001)
2. Objekt B: lon=59.1955, lat=17.6254 (Kund K001) – inom 30m
3. Objekt C: lon=59.1960, lat=17.6260 (Kund K001) – utanför 30m
4. Objekt D: lon=59.1955, lat=17.6253 (Kund K002) – samma plats, ANNAN kund

VERIFIERA: A + B grupperas som ett jobb

VERIFIERA: C är separat jobb

VERIFIERA: D är separat jobb (annan kund, trots samma plats)

Test E6: Tidskonsolidering

SCENARIO 1: Tidsfönster "hela maj, torsdagar"

VERIFIERA: Konsolideras till första torsdagen i maj

VERIFIERA: Markering att underliggande tidsvillkor finns

SCENARIO 2: Tidsfönster "torsdag 09:00-11:30"

VERIFIERA: Specifik tid bevaras

SCENARIO 3: Tidsfönster "januari-februari"

VERIFIERA: Konsolideras till startdatum av perioden

Test E7: Fakturagruppering

STEG:

1. Orderkoncept skapar uppgifter för 3 objekt (avfallsrum)
2. Varje uppgift har beställningsreferens från metadata
3. VERIFIERA: Faktura grupperas per objekt och referens
4. VERIFIERA: Korrekt format: "Återvinningsrum 22-30: Tvätt matavfallskärl 20 st, Tvättrum 1 st, Dörrkontroll OK"

Test E8: Dynamisk uppdatering av framtida uppgifter

STEG:

1. Orderkoncept OC-001 har genererat 6 uppgifter (3 perioder × 2 objekt)
2. 2 uppgifter är utförda, 4 är ogjorda
3. Utförare uppdaterar antal_kärl 10→15 på objekt 001
4. VERIFIERA: De 2 utförda uppgifterna ÄNDRAS INTE
5. VERIFIERA: De ogjorda uppgifterna för objekt 001 uppdateras till 15
6. VERIFIERA: Uppgifter för objekt 002 ÄNDRAS INTE

3. Edge Cases & Felhantering

Test F1: Tomt kluster

Orderkoncept pekas in på kluster utan matchande objekt
FÖRVÄNTAT: Inga uppgifter skapas, informativt meddelande

Test F2: Artikel utan haka-på villkor

Artikel utan metadatavillkor i orderkoncept
FÖRVÄNTAT: Artikel behandlas som "vanlig order" med manuellt antal

Test F3: Objekt utan geografisk position

Uppgift kopplas till objekt utan lon/lat
FÖRVÄNTAT: Uppgift skapas men kan inte grupperas geografiskt
Grovplanering visar uppgiften utan plats

Test F4: Cirkulär objekthierarki

Försök sätta objekt A som barn till B och B som barn till A
FÖRVÄNTAT: Blockerat på databasnivå

Test F5: Fortnox-synk misslyckande

Skapa kund/artikel som INTE finns i Fortnox
FÖRVÄNTAT: Tydligt felmeddelande, operation blockeras

Test F6: Metadatafält-borttagning med existerande data

Försök ta bort metadatadefinition som används av 100 objekt
FÖRVÄNTAT: Varning + bekräftelse krävs ELLER blockerat

4. Acceptanskriterier

Godkänt kräver att ALLA följande är sanna:

#	Kriterie	Status
1	Alla 13+ tabeller existerar med korrekta fält och relationer	☐
2	Multi-tenant isolering: 0% dataläckage mellan företag	☐
3	Alla FK-constraints fungerar (referential integrity)	☐
4	Komplett flöde Order→Uppgift→Utförande→Fakturerering fungerar	☐
5	Haka-på logik matchar Mats beskrivning (villkor, multipla kriterier)	☐
6	3-steps kvantitetsresolution (order-koncept→metadata→utförare) fungerar	☐
7	Dynamisk uppdatering av framtida uppgifter vid metadata-ändring	☐
8	30m jobbgruppering med kundfiltrering fungerar	☐
9	Strukturartiklar genererar uppgifter för alla underartiklar	☐
10	Metadata-hierarki (t.ex. Kontakt → underfält) fungerar	☐
11	Metadata-arv neråt i kluster fungerar	☐
12	Tidskonsolidering visar rimliga resultat	☐
13	Prestanda: alla kritiska queries < 3 sekunder med 100k objekt	☐
14		☐

#	Kriterie	Status
	Inga N+1 query-problem i huvudflöden	
15	Orphaned records förhindras	☐
16	Audit trail (loggning) vid metadata-uppdateringar	☐

5. Regressionstest

5.1 Automatiserade tester (ska köras vid varje schemaändring)

- Alla CRUD-tester per entitet
- Multi-tenant isoleringstester
- Relationsintegritetstester
- Komplet flöde (E1) som end-to-end test

5.2 Manuella tester (vid större ändringar)

- Kundvy: alla associationer visas korrekt
- Grovplanering: lista med korrekt data
- Orderkoncept: bekräftelseflöde med metadata-matchning
- Fakturering: korrekt gruppering

5.3 Databasmigrerings-regression

FÖR VARJE migrering:

1. Ta backup av testdatabas
2. Kör migrering
3. Kör ALLA automatiserade tester
4. VERIFIERA: Befintlig data är intakt
5. VERIFIERA: Nya strukturer fungerar
6. Om test misslyckas: rollback till backup

6. Testprioriteringsordning

Prioritet	Test	Motivering
 P1	B1-B4 (Multi-tenant)	Dataläckage = kritisk säkerhetsbrist
 P1	E1 (Komplett flöde)	Kärnfunktionalitet
 P1	E3-E4 (Haka-på + kvantitet)	Mats centrala logik
 P2	A1-A4 (Schema-validering)	Strukturell grund
 P2	C1-C6 (Relationsintegritet)	Datakonsistens
 P2	E8 (Dynamisk uppdatering)	Affärskritisk
 P3	D1-D4 (Prestanda)	Skalbarhet
 P3	E5-E7 (Jobbgruppering, tid, faktura)	Avancerad funktionalitet
 P4	F1-F6 (Edge cases)	Robusthet